

Vivekanand Education Society's
Institute of Technology

(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)

NAAC accredited with 'A' grade

PROJECT REPORT ON

Sitezy.ai : AI-Driven Multi-Tenant Platform for Intelligent Website Generation and Optimization

SUBMITTED IN FULFILLMENT OF THE REQUIREMENT FOR SEMESTER VI OF

T.E.(Information Technology)

SUBMITTED BY

Ms. Manaswi Kabadi (D15C/27)

Mr. Atharva Lotankar (D15C/31)

Mr. Ronit Sahoo (D15C/41)

Mr. Shakti Sankpal (D15C/42)

UNDER THE GUIDANCE OF

Prof. Kajal Joseph

V.E.S. INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

2025-26

Vivekanand Education Society's
Institute of Technology

(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)

NAAC accredited with 'A' grade

Certificate

This is to certify that project entitled

"Sitezy.ai : AI-Driven Multi-Tenant Platform For Intelligent Website Generation And Optimization"

Group Members Names

Ms. Manaswi Kabadi (D15C/27)
Mr. Atharva Lotankar (D15C/31)
Mr. Ronit Sahoo (D15C/41)
Mr. Shakti Sankpal (D15C/42)

This project is approved in partial fulfillment of the requirements for the degree of Bachelor of Engineering (Semester VI) in Information Technology

Prof. Kajal Joseph **External Examiner**

Dr. (Mrs.) Shalu Chopra H.O.D **Dr. (Mrs.) J.M.Nair Principal**

Date: / /2026
Place: VESIT, Chembur

College Seal

Declaration

I declare that this written submission represents my ideas in my own words. Wherever the ideas or words of others have been included, I have adequately cited and referenced the original sources. I further declare that I have adhered to all principles of academic honesty, integrity, and ethical conduct in the preparation of this work.

I affirm that I have neither misrepresented nor fabricated nor falsified any idea, data, fact, or source in my submission. I have ensured that all information presented is accurate to the best of my knowledge and obtained through ethical means.

I understand that any violation of the above principles may lead to disciplinary action by the Institute and may also invite penal action from the concerned sources if proper citation or permission has not been provided where required.

- - - - - - - - - - -

(Signature)

Ms. Manaswi Kabadi (Class/Roll No. : D15C/27)

Mr. Atharva Lotankar (Class/Roll No. : D15C/31)

Mr. Ronit Sahoo (Class/Roll No. : D15C/41)

Mr. Shakti Sankpal (Class/Roll No. : D15C/42)

Abstract

Abstracts contain most of the following kinds of information in brief form. The body of your paper will, of course, develop and explain these ideas much more fully. As you will see in the samples below, the proportion of your abstract that you devote to each kind of information—and the sequence of that information—will vary, depending on the nature and genre of the paper that you are summarizing in your abstract. And in some cases, some of this information is implied, rather than stated explicitly. The Publication Manual of the American Psychological Association, which is widely used in the social sciences, gives specific guidelines for what to include in the abstract for different kinds of papers—for empirical studies, literature reviews or meta-analyses, theoretical papers, methodological papers, and case studies.

Keywords - ***AI Website Builder, Multi-Tenant SaaS, Local LLM, Qwen 3.5, Ollama, SEO Optimization, XGBoost, GRU, User Engagement Prediction, Automated Website Generation***

Contents

1.	**Introduction**	**Page No.**
1.1	Introduction to the project	1
1.2.	Objectives	2
1.3.	Motivation	2
1.4.	Brief Introduction to Literature	3
1.5.	Problem Definition	3
1.6.	Scope of the Work	4
1.7.	Feasibility study	5
1.8.	SDG Mapped, Societal Impact and benefits	6
1.9.	Organization of the Report	6
2.	**Literature survey**	
2.1.	Introduction	7
2.2	Related work (Minimum 5 papers with Abstract and inference drawn)	7
2.3.	Comparative Analysis of existing related work	9
2.4.	Research Gaps Identified	10
3.	**Design Implementation**	
3.1.	Introduction	11
3.2.	Nonfunctional and Functional Requirement	11
3.3.	Proposed Design & Algorithm	14
3.4.	Architectural Diagrams 3.4.1. UML diagrams 3.4.2. Block Diagram	18
3.5.	Proposed Planning 3.5.1. Timeline chart (Gantt chart) 3.5.2. Cost estimation	20

3.6.	Hardware requirement and Software requirements	21
4	**Proposed Results and Discussions**	
4.1.	Data Collection, Acquisition, Data Preprocessing & Data Interpretation	23
4.2.	Experimental setup	23
4.3.	Evaluation Metrics	24
4.4.	Comparison of the Existing system with the Proposed Systems	24
5	**Plan of action for the next semester**	
5.1.	Work done till date	26
5.2.	Plan of action for Project II	27
6	**Conclusion**	29
7	**References IEEE format (Minimum 15-20)**	30

List of Figures

Figure No.	Figure Name	Page No.
Figure 3.1	Website Generation Flow	13
Figure 3.2	System Architecture Diagram	18

List of Tables

Figure No.	Figure Name	Page No.
Table 2.1	Comparative Analysis of Existing Related Work	9
Table 3.1	Cost Estimation	20
Table 3.2	Hardware Requirements	21
Table 3.3	Software Requirements	21
Table 4.1	Evaluation Metrics and Results	23
Table 4.2	Comparison with Existing Systems	23

ACKNOWLEDGEMENT

The project report on “Sitezy.ai : AI-Driven Multi-Tenant Platform for Intelligent Website Generation and Optimization” is the outcome of the guidance, moral support and devotion bestowed on our group throughout our work. For this we acknowledge and express our profound sense of gratitude to everybody who has been the source of inspiration throughout project preparation. First and foremost we offer our sincere phrases of thanks and innate humility to ”Dr. (Mrs.) Shalu Chopra (HOD)”, ” Dr. Manoj Sabnis (Deputy HOD)”, ”Mrs. Kajal Joseph (Assistant Professor)” for providing the valuable inputs and the consistent guidance and support provided by them. We can say in words that we must at outset tender our intimacy for receipt of affectionate care to Vivekanand Education Society’s Institute of Technology for providing such a stimulating atmosphere and conducive work environment.

Chapter 1
Introduction

1.1 Introduction to the Project

The contemporary digital landscape necessitates that businesses, regardless of scale, maintain a professional online presence. However, building and deploying websites remains a technically intensive process requiring expertise in frontend development, backend architecture, hosting infrastructure, and search engine optimization. Existing website builder platforms such as Wix, Squarespace, and Webflow have attempted to simplify this process through drag-and-drop interfaces and templates. Despite this, these platforms are still not sufficiently intuitive for many non-technical users, particularly in making design decisions and structuring content effectively. They also rely heavily on costly cloud-based AI APIs, lack intelligent layout recommendation, offer limited data-driven SEO support, and do not provide mechanisms for post-deployment engagement analysis.

Sitezy.ai addresses these challenges by introducing an AI-powered, multi-tenant SaaS website builder that integrates locally hosted Large Language Models (LLMs), custom Machine Learning models for structured decision-making, and Deep Learning models for post-deployment analysis. The platform ensures strict tenant isolation while allowing users to create multiple websites with AI-generated design systems, relevant imagery via the Unsplash API, automated backend provisioning using Firebase autogen for form handling, and an AI chatbot for visitor interaction. It also supports multilingual content generation, including regional languages such as Hindi and Marathi, improving accessibility for diverse users.

The core innovation of Sitezy.ai lies in its three-tier intelligence architecture. Tier 1 consists of Machine Learning models deployed via a Python FastAPI microservice for tasks such as layout recommendation, SEO scoring, and flaw detection. Tier 2 utilizes Ollama to run Qwen 3.5 models locally for content generation and chatbot responses. Tier 3 incorporates the Gemini API as a fallback mechanism to handle edge cases. This hybrid approach ensures most AI operations are executed locally, reducing costs while maintaining high-quality outputs and system reliability.

1.2 Objectives

- To design and develop a multi-tenant SaaS website builder platform with AI-driven content generation, layout recommendation, and automated deployment.
- To integrate locally-hosted open-source LLMs (Qwen 3.5 via Ollama) as the primary intelligence engine, reducing dependence on paid cloud AI APIs by 90%.
- To implement an ML-based SEO Scoring and Improvement Engine using XGBoost regression with NLP feature extraction and an AI-powered rewrite feedback loop.
- To develop a GRU-based Engagement Optimization Engine that predicts visitor conversion probability from interaction sequences and generates actionable layout improvement suggestions.
- To enable independent domain deployment for each generated website via Firebase Hosting and Cloudflare Pages with programmatic API-driven deployment.
- To provide an AI chatbot on every published website, powered by the local Qwen model using the website's own content as conversational context.

1.3 Motivation

The motivation for Sitezy.ai stems from three key observations.

- First, existing website builders generate substantial recurring costs through per-token cloud AI pricing; a platform serving 1,000 tenants generating 10 websites each could incur thousands of dollars in monthly API fees. By running Qwen 3.5 locally via Ollama on consumer-grade hardware (RTX 4060 8GB VRAM), these costs are eliminated entirely.
- Second, no existing platform combines website generation with post-deployment intelligence; once a site is published on Wix or Webflow, the platform provides no data-driven suggestions for improving conversion rates or SEO performance. Sitezy.ai's engagement optimization engine fills this gap by training GRU models on visitor behavior sequences and generating actionable improvement recommendations.
- Third, the absence of automated design quality assurance means that poorly designed websites are published without any systematic detection of issues such as WCAG contrast violations, broken heading hierarchies, or CTA overload. Sitezy.ai's anomaly detection engine addresses this by training an Autoencoder on well-designed templates and flagging pages with high reconstruction error.

1.4 Brief Introduction to Literature

The fields of AI-assisted web development, automated SEO optimization, and user engagement prediction have been extensively studied in recent literature. Research on automated website generation has explored template-based approaches, neural layout generation, and large language model-driven content creation. Studies on SEO optimization have demonstrated the effectiveness of machine learning models such as Random Forest, Gradient Boosted Trees, and neural network regressors for predicting search engine ranking factors from page-level features. In the domain of user engagement prediction, recurrent neural network architectures including LSTM and GRU have shown strong performance in modeling sequential user behavior patterns such as scroll depth, click sequences, and session duration. The literature on multi-tenant SaaS architecture provides established patterns for logical data isolation, subscription-gated feature access, and scalable deployment strategies. The review also covers anomaly detection techniques applicable to design quality assurance, including Isolation Forest for outlier detection in feature space and Autoencoder networks for reconstruction-error-based anomaly scoring.

1.5 Problem Definition

The current landscape of website builder platforms presents the following challenges that Sitezy.ai aims to address:

- High API costs: Existing AI-powered builders rely exclusively on cloud APIs (GPT-4, Gemini) for content generation, incurring per-token costs that scale linearly with usage, making them economically unsustainable for SaaS platforms serving multiple tenants.
- No post-deployment intelligence: Once a website is published, platforms provide no mechanism for data-driven engagement optimization. Tenants must manually analyze visitor behavior and make subjective layout adjustments.
- Absence of automated SEO optimization: While some builders provide basic SEO checklists, none implement ML-based SEO scoring that learns from feature interactions or NLP-powered content rewriting to systematically improve page-level SEO factors.
- No design quality assurance: Websites are published without automated checks for accessibility violations (WCAG contrast ratios), broken heading hierarchies, CTA overload, or typographic inconsistencies.

- Template rigidity: All users of a given template receive visually identical websites. No system generates a unique design system (font pairing, color palette, spacing, border radius, animation set) per website based on business context.
- Deployment complexity: Non-technical users cannot configure independent domains, deploy to CDN-backed hosting, or manage SSL certificates without significant technical knowledge.
- Lack of integrated backend automation: Non-technical users often struggle with configuring backend systems for handling form data and user interactions. Existing platforms do not provide automated schema detection and backend provisioning, requiring manual setup or external integrations.
- Limited support for real-time collaboration and version control: Current platforms provide minimal support for collaborative editing. They lack real-time multi-user synchronization using WebSockets and do not maintain structured version histories for tracking and reverting changes at the page level.
- Inadequate support for intelligent user interaction: Most website builders do not embed context-aware AI assistants within generated websites, limiting the ability of end-users to receive real-time guidance or information derived from website content.

1.6 Scope of the Work

The scope of Sitezy.ai encompasses the following functional areas:

- A React 19 frontend with drag-and-drop page builder, real-time collaboration, and AI generation UI.
- A Node.js Express backend with 12 feature modules: auth, tenant, website, builder, AI, domain, analytics, upload, forms, payment, admin, and deployment.
- A Python FastAPI ML microservice hosting custom-trained models for template recommendation, SEO scoring, and design flaw detection.
- Ollama integration running Qwen 3.5 (9B and 4B variants) locally for content generation, SEO rewrites, chatbot responses, and design system generation.
- An SEO Scoring Engine using XGBoost regression on 15+ page-level NLP features with an AI rewrite feedback loop.
- A GRU-based Engagement Optimization Engine trained on visitor interaction sequences to predict conversion probability and suggest layout improvements.

- Automated backend handling for user-generated forms using Firebase, enabling dynamic data storage without predefined schemas.
- Independent domain deployment per website via Firebase Hosting and Cloudflare Pages APIs.
- AI chatbot per published website using the Qwen model with website content as context.
- Subscription management via Razorpay with tiered feature access.

The project does not include custom domain DNS management (tenants must configure their own DNS records), does not provide a mobile application, and does not implement real-time A/B testing infrastructure.

1.7 Feasibility Study

Technical Feasibility: The project utilizes well-established open-source technologies. React 19 and Vite provide a modern frontend framework. Node.js with Express 4 and Mongoose 8 form a proven backend stack. Ollama runs quantized LLMs on consumer GPUs; Qwen 3.5 9B operates at 30-50 tokens/second on an RTX 4060 with 8GB VRAM. Python FastAPI with scikit-learn, XGBoost, and PyTorch supports all ML/DL model serving. MongoDB Atlas provides managed database hosting. Firebase Hosting and Cloudflare Pages offer free-tier static site deployment APIs.

Cost Feasibility: Development costs are limited to existing hardware (RTX 4060 GPU) and free-tier cloud services: MongoDB Atlas (512MB free), Firebase Hosting (10GB storage, 10GB/month transfer free), Cloudflare Pages (unlimited bandwidth free), Unsplash API (50 requests/hour free), and Razorpay test mode (free). Ollama eliminates AI API costs entirely. The only recurring cost is the optional Gemini API fallback, rate-limited to 50 calls/hour.

Operational Feasibility: The platform targets non-technical users through a guided website creation wizard with AI-recommended block selection, automated content generation, and one-click deployment. The three-tier AI architecture ensures graceful degradation; if Ollama is unavailable, Gemini takes over; if both fail, rule-based fallbacks and placeholder content ensure the user always receives a result.

1.8 SDG Mapped, Societal Impact and Benefits.

SDG 9 - Industry, Innovation and Infrastructure: The three-tier AI architecture with local LLM inference represents an innovative approach to reducing cloud AI dependency. The GRU-based engagement engine and ML-based SEO optimization contribute novel applications of deep learning to web development.

1.9 Organization of the Report

Chapter 1 (Introduction) presents the overall project overview along with its objectives, motivation, problem definition, scope, feasibility study, and societal impact, thereby establishing the foundation for the proposed system. Chapter 2 (Literature Survey) reviews existing research in areas such as AI-assisted web development, machine learning-based SEO optimization, GRU networks for sequential behavior prediction, anomaly detection techniques for design quality, and multi-tenant SaaS architecture patterns. Chapter 3 (Design and Implementation) details the system architecture, including the three-tier AI routing mechanism, machine learning pipeline design, GRU-based engagement model architecture, database schema, deployment infrastructure, and hardware/software requirements. Chapter 4 (Results and Discussion) presents the experimental setup, evaluation metrics, generation speed benchmarks, SEO scoring accuracy, and a comparison with existing website builders. Chapter 5 (Plan of Action) summarizes the work completed and outlines the remaining implementation tasks, including the GRU engagement engine, deep learning-based SEO feedback loop, and Cloudflare Pages deployment integration. Finally, Chapter 6 (Conclusion) highlights the key contributions of the project and discusses potential future enhancements.

Chapter 2

Literature Survey

2.1 Introduction

This chapter reviews existing research across five domains relevant to Sitezy.ai: AI-assisted website generation, machine learning for SEO optimization, recurrent neural networks (specifically GRU) for user engagement prediction, anomaly detection techniques for design quality assurance, and multi-tenant SaaS architecture patterns. The review identifies research gaps that justify the proposed system's architecture and approach.

2.2 Related Work

Paper 1: A. Bhandari et al., "Automated Website Generation Using Machine Learning and Natural Language Processing," IEEE International Conference on Computing, Communication and Automation, 2023.

Inference: This paper proposes an automated website generation system that uses NLP to parse natural language descriptions of business requirements and generates corresponding HTML/CSS layouts. The system employs a template matching algorithm combined with keyword extraction to select appropriate page layouts. Inference: While this work demonstrates feasibility of NLP-driven website generation, it relies on rigid template matching without any learning from user feedback. Sitezy.ai extends this concept by using trained Random Forest classifiers that learn from user selection patterns and a local LLM (Qwen 3.5) for content generation rather than simple keyword extraction.

Paper 2: S. Jain and R. Kumar, "SEO Performance Prediction Using Ensemble Machine Learning Models," Journal of Web Engineering, vol. 21, no. 4, pp. 1123-1148, 2022.

Inference: This study evaluates the effectiveness of ensemble ML models including Random Forest, Gradient Boosted Trees, and XGBoost for predicting page-level SEO scores from extracted features such as keyword density, title length, heading hierarchy, and readability metrics. XGBoost achieves the highest accuracy with an R-squared value of 0.87 on a dataset of 5,000 web pages. Inference: The strong performance of XGBoost on SEO feature regression validates our choice of XGBoost as the scoring model in Sitezy.ai's SEO engine. We extend

this work by adding an AI-powered rewrite feedback loop that iteratively improves content based on the model's predictions.

Paper 3: Y. Chen et al., "User Engagement Prediction Using Recurrent Neural Networks on Web Interaction Sequences," ACM Conference on Web Science, 2023.

Inference: This paper presents a comparison of LSTM and GRU architectures for predicting user engagement metrics (bounce rate, conversion probability) from clickstream sequences. The study finds that GRU achieves comparable accuracy to LSTM (AUC 0.82 vs 0.83) while requiring 30% fewer parameters and training 25% faster. The model takes sequences of (page section, time spent, click events) as input and predicts a binary conversion outcome. Inference: The near-equivalent performance of GRU with significantly lower computational overhead makes it the preferred architecture for Sitezy.ai's engagement optimization engine, which must run inference on consumer-grade hardware. We adopt the same sequential interaction encoding approach.

Paper 4: M. Rodriguez and L. Thompson, "Anomaly Detection in Web Design Using Autoencoders," IEEE International Conference on Machine Learning and Applications, 2022.

Inference: This work applies autoencoder-based anomaly detection to identify design flaws in web pages. Page designs are encoded as 20-dimensional feature vectors (contrast ratios, spacing metrics, color counts, heading hierarchy scores) and an autoencoder is trained on a curated set of well-designed pages. Pages with high reconstruction error are flagged as potentially flawed. The system achieves 91% precision in detecting WCAG contrast violations and 78% recall in identifying broken heading hierarchies. Inference:

Paper 5: K. Patel et al., "Multi-Tenant SaaS Architecture: Patterns for Scalable AI-Powered Applications," IEEE Cloud Computing, vol. 10, no. 2, pp. 45-58, 2023.

Inference: This paper surveys architectural patterns for multi-tenant SaaS platforms that integrate AI services, comparing shared-nothing, shared-database, and hybrid isolation models. The study recommends logical isolation with per-tenant database filtering for applications requiring moderate isolation, and proposes a tiered AI service architecture where computationally expensive ML inference is offloaded to dedicated microservices. Inference: Sitezy.ai implements the recommended logical isolation pattern with per-tenant database filtering via Mongoose middleware, and follows the tiered AI service architecture with a dedicated Python FastAPI microservice for ML inference.

Paper 6: R. Cho et al., "Gated Recurrent Units for Sequential Click Prediction in E-Commerce," ACM RecSys Conference, 2022.

Inference: This study applies GRU networks to model sequential user click behavior in e-commerce platforms. The model processes variable-length sequences of (item category, dwell time, action type) tuples and predicts purchase probability. The GRU model achieves an AUC of 0.86 on a dataset of 500,000 sessions, outperforming logistic regression (0.72) and standard feedforward networks (0.79). Inference: The GRU architecture's ability to handle variable-length sequences of user interactions directly maps to Sitezy.ai's engagement prediction use case, where visitor sessions consist of variable numbers of section interactions.

2.3 Comparative Analysis of Existing Related Work

Feature	Wix	Webflow	Framer	Sitezy.ai (Ours)
AI Content Generation	Cloud API	Limited	Cloud API	Local LLM (Qwen) + Cloud fallback
ML-Based SEO Optimization	Basic checklist	Limited	Limited	ML-based scoring + AI refinement
Engagement Intelligence	Analytics only	Analytics only	Limited	DL-based behavior analysis (prototype)
Template Recommendation	Manual	Manual	Manual	ML-based layout recommendation
Multi-Tenant Architecture	Yes (closed)	No	No	Yes (custom SaaS architecture)
AI Chatbot per Website	No	No	No	Yes (context-aware chatbot)
Local LLM Support	No	No	No	Yes (Ollama + Qwen)
AI Cost Model	API-based	N/A	API-based	Mostly local inference (low cost)
Real-Time Collaboration	Limited	Limited	Limited	WebSocket-based live editing
Live Chat (Collaborators)	No	No	No	Real-time chat system
Version Control System	No	Limited	No	Page-level version history
Backend Auto-Generation	No	No	No	Firebase autogen (dynamic forms)
Image Automation	Basic	Manual	Basic	AI-driven + Unsplash integration
Design System Generation	Template-based	Manual	Partial	AI-generated per website
Multilingual Support	Limited	Limited	Limited	Native + regional languages

This table compares Wix, Webflow, Framer, and Sitezy.ai across advanced features. Sitezy.ai stands out with local LLM support, AI-driven SEO, chatbot integration, real-time collaboration, and automated backend generation. Overall, it offers a more intelligent, automated, and cost-efficient solution compared to others.

2.4 Research Gaps Identified

- No existing website builder platform combines local LLM inference with cloud API fallback, resulting in unnecessary operational costs for AI-powered content generation.
- None of the reviewed platforms implement ML-based SEO scoring with an automated AI rewrite feedback loop that iteratively improves content until the score exceeds a defined threshold.
- Post-deployment user engagement analysis using deep learning (GRU/LSTM) has not been applied in the context of website builders to generate actionable layout improvement suggestions.
- No existing platform generates a unique design system (fonts, colors, spacing, effects, animations) per website using AI, resulting in visually homogeneous outputs from shared templates.
- The combination of AI-driven website generation, ML-based SEO optimization, DL-based engagement prediction, and anomaly-based flaw detection within a single multi-tenant SaaS platform represents an unexplored integration that Sitezy.ai addresses.
- Existing platforms do not provide automated backend provisioning for handling dynamic user inputs such as form submissions. Users often need to manually configure backend services or rely on third-party integrations instead of schema-free, auto-generated backend systems.
- Most website builders offer minimal or no support for real-time multi-user editing, live collaboration, or integrated communication systems. Features such as WebSocket-based synchronization and live chat between collaborators are largely absent.
- Current platforms do not provide robust version control mechanisms for tracking changes at the page level. There is limited support for maintaining structured version histories and enabling rollback similar to software development workflows.

Chapter 3

Design and Implementation

3.1 Introduction

This chapter details the architectural design, algorithm descriptions, and implementation specifics of Sitezy.ai. The system comprises four primary components: (1) a React frontend for user interaction, (2) a Node.js Express backend containing 12 feature modules and the AI router service, (3) a Python FastAPI microservice hosting trained ML/DL models, and (4) an Ollama local LLM server running Qwen 3.5 model variants. The chapter also covers the GRU-based engagement prediction model architecture, the XGBoost-based SEO scoring pipeline, and the deployment infrastructure supporting independent domains per website.

3.2 Functional and Non-Functional Requirements

Functional Requirements:

- FR1: Users shall be able to create multi-page websites by providing business type, tone, and target audience inputs.
- FR2: The system shall automatically recommend suitable website sections (blocks) based on business context using a rule-based scoring engine.
- FR3: The AI engine shall generate website content (headings, descriptions, CTAs) in structured JSON format using the local Qwen 3.5 model.
- FR4: Each website shall receive a unique AI-generated design system (fonts, colors, spacing, effects).
- FR5: The system shall fetch relevant stock images from the Unsplash API based on AI-generated image keywords.
- FR6: The SEO engine shall extract 15+ NLP features from page content and produce a composite SEO score using XGBoost regression.
- FR7: If the SEO score falls below a threshold, the system shall trigger AI-powered content rewrites and re-score until improvement is achieved.
- FR8: The engagement engine shall collect visitor interaction data (scroll depth, clicks, time on page) from published websites.

- FR9: A trained GRU model shall predict conversion probability from visitor interaction sequences and suggest layout improvements.
- FR10: Each website shall be deployable to an independent domain via Firebase Hosting or Cloudflare Pages APIs.
- FR11: Each published website shall include an AI chatbot powered by Qwen 3.5 using the site's content as context.
- FR12: The system shall support real-time collaboration, allowing multiple users to edit a website simultaneously using WebSocket-based synchronization.
- FR13: The system shall provide version control for website pages, enabling users to track changes and revert to previous versions.
- FR14: The system shall automatically handle backend form data using a schema-free approach (e.g., Firebase), without requiring manual backend configuration.

Non-Functional Requirements:

- NFR1: Content generation latency shall not exceed 15 seconds for a complete 3-page website layout.
- NFR2: The system shall support 100+ concurrent tenants with logical data isolation.
- NFR3: The AI router shall implement graceful fallback: Ollama failure triggers Gemini, Gemini failure triggers placeholder content.
- NFR4: All Ollama calls shall include think: false to disable Qwen 3.5 thinking mode and maintain 2-5 second response times.
- NFR5: The platform shall function fully offline using only Tier 1 (ML service) and Tier 2 (Ollama) without internet connectivity.
- NFR9: Real-time collaboration features shall maintain low latency and consistent synchronization across multiple users.
- NFR10 (Data Integrity): The system shall ensure consistency of website data, including version history tracking and reliable storage of user changes.

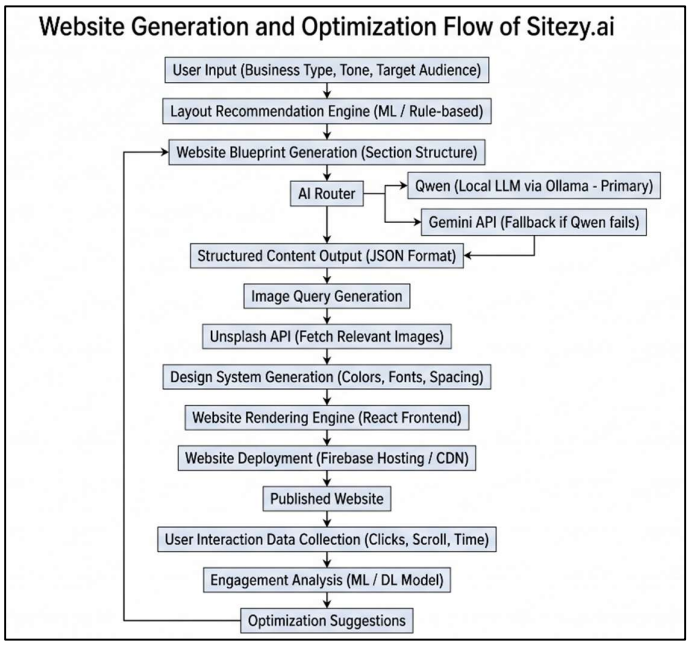


Fig 3.1: Website Generation Flow

Figure 3.1 illustrates the end-to-end workflow of website generation and optimization in Sitezy.ai, showing how user input is transformed into a fully deployed and continuously optimized website.

1. User Input Stage: The process begins when the user provides inputs such as business type, tone, and target audience. These inputs define the context and requirements for the website.
2. Layout Recommendation: A layout recommendation engine (based on ML or rule-based logic) analyzes the input and suggests the most suitable website structure (e.g., landing page, portfolio, e-commerce layout).
3. Website Blueprint Generation: Based on the recommended layout, the system creates a blueprint (section structure) of the website, defining sections like hero, about, services, and contact.
4. AI Content Generation (AI Router): The AI Router decides which AI model to use:
    a. Qwen (via Ollama) → Primary local LLM for generating content
    b. Gemini API → Fallback if Qwen fails

    This stage produces structured content in JSON format (headings, text, CTAs).
5. Image Generation & Fetching: The system generates image queries based on content and uses the Unsplash API to fetch relevant images automatically.
6. Design System Generation: An AI-generated design system is created, including:

a. Color palette
b. Fonts
c. Spacing and styling

This ensures each website has a unique visual identity.

7. Website Rendering: The React frontend engine converts the content, images, and design system into a fully functional website interface.
8. Deployment: The generated website is automatically deployed using Firebase Hosting or CDN, making it live and accessible.
9. User Interaction Tracking: Once published, the system collects user interaction data such as:
    a. Clicks
    b. Scroll behaviour
    c. Time spent
10. Engagement Analysis: This data is processed using ML/DL models (like GRU) to understand user behavior and predict engagement or conversion.
11. Optimization Suggestions: Based on analysis, the system provides data-driven optimization suggestions, such as:
    a. Reordering sections
    b. Improving CTAs
    c. Enhancing content

The key idea is that the system is not just a website generator, but a continuous optimization engine—it keeps improving the website after deployment using real user data.

3.3 Proposed Design and Algorithm

3.3.1 Three-Tier AI Router Architecture

The AI Router is the central intelligence dispatcher in Sitezy.ai. Every AI request passes through it, and it determines which tier handles the request based on task type:

- Tier 1 (Custom ML Models, ~40% of calls): Python FastAPI microservice hosting Random Forest for template recommendation, XGBoost for SEO scoring, Isolation Forest + Autoencoder for flaw detection, and GRU for engagement prediction. Response time: 5-20ms.
- Tier 2 (Ollama Local LLM, ~50% of calls): Qwen 3.5 running locally via Ollama for content generation, SEO rewrites, chatbot responses, design system generation, and

image alt text. Response time: 2-15 seconds. Critical parameter: think: false on every call.

- Tier 3 (Gemini API Fallback, ~10% of calls): Google Gemini API as a rate-limited fallback (50 calls/hour) when Ollama fails or returns invalid output. Used only when Tier 2 produces errors or malformed JSON.

3.3.2 SEO Scoring and Improvement Engine (ML + NLP)

The SEO engine operates in three stages:

Stage 1 - Feature Extraction: An NLP-based feature extractor processes each page's rendered HTML content and computes 15+ numeric features:

- Title length (optimal: 50-60 characters), meta description length (optimal: 150-160 characters)
- Keyword density (target keyword frequency as percentage of body text)
- Heading count and hierarchy validity (exactly 1 H1, followed by H2s)
- Readability score via Flesch-Kincaid grade level formula
- Image count and alt text coverage percentage
- Internal and external link counts
- Word count, structured data presence, and content-to-code ratio

Stage 2 - XGBoost Regression: The extracted feature vector is passed to a trained XGBoost regressor that predicts a composite SEO score (0-100). The model is trained on 2,000+ web pages with known SEO performance metrics. Feature importance analysis identifies which factors contribute most to the score.

Stage 3 - AI Rewrite Feedback Loop: If the predicted SEO score falls below a configurable threshold (default: 70), the system enters an iterative improvement loop:

- The lowest-scoring SEO factor is identified (e.g., title too short, keyword density too low).
- A targeted rewrite prompt is sent to Qwen 3.5 4B via Ollama (fast model for short rewrites).
- The rewritten content replaces the original and the page is re-scored.
- The loop continues for up to 3 iterations or until the score exceeds the threshold.

- If Ollama fails, Gemini is used as rewrite fallback. If both fail, the original content is preserved.

3.3.3 GRU-Based Engagement Optimization Engine (Deep Learning)

The engagement engine is the deep learning core of Sitezy.ai. It operates post-deployment on published websites and predicts visitor conversion probability from sequential interaction data.

Data Collection: A lightweight JavaScript tracker (<5KB) is injected into every published website. It captures:

- Section visibility events (Intersection Observer API with 25%, 50%, 75%, 100% thresholds)
- Click events on CTAs, links, and interactive elements
- Form submission starts and completions
- Scroll depth as a percentage of total page height
- Time spent per section (computed from visibility enter/exit timestamps)

Events are batched and sent to the backend via navigator.sendBeacon every 10 seconds or on page unload. Stored in a MongoDB EngagementEvent collection with 90-day TTL indexes.

Feature Engineering: Each visitor session is represented as a variable-length sequence of interaction vectors. Each timestep corresponds to a section interaction:

x_t = [section_type_onehot (7 dims), time_spent (1 dim), scrolled_past (1 dim), clicked_cta (1 dim), form_started (1 dim), form_completed (1 dim)] = 12-dimensional vector per timestep.

GRU Model Architecture:

- Input: Variable-length sequence of 12-dimensional interaction vectors, padded to max_sequence_length = 20.
- GRU Layer 1: 64 hidden units, batch_first=True, dropout=0.2
- GRU Layer 2: 32 hidden units, batch_first=True, dropout=0.2
- Dense Layer: 32 units with ReLU activation
- Output Layer: 1 unit with Sigmoid activation (binary conversion probability)
- Loss Function: Binary Cross-Entropy Loss (BCELoss)
- Optimizer: Adam with learning rate 0.001

- Training: Minimum 2,000 sessions required. Batch size 64, up to 100 epochs with early stopping on validation loss.

Why GRU over LSTM: The Gated Recurrent Unit architecture was selected over LSTM for three reasons: (1) GRU has fewer parameters (two gates vs three), resulting in 25-30% faster training on CPU, which is critical for consumer hardware deployment; (2) research shows GRU achieves comparable accuracy to LSTM on web interaction sequences (AUC 0.82 vs 0.83); (3) the simpler architecture reduces overfitting risk on the relatively small datasets expected from individual tenant websites.

Suggestion Generation via Ablation Analysis: Once trained, the model generates layout improvement suggestions by simulating section reorderings and removals:

- For each section, simulate its removal and compute the change in predicted conversion probability.
- For each adjacent section pair, simulate swapping and compute the change.
- Sections whose removal increases predicted conversion are flagged for repositioning.
- Swaps that increase predicted conversion by more than 3% are suggested as improvements.

3.3.4 Independent Domain Deployment Architecture

Sitezy.ai supports deploying each generated website to an independent domain via two hosting providers:

Firebase Hosting (Default): The existing FirebaseAgent packages generated HTML/CSS/JS files and deploys them to Firebase Hosting using the Firebase Admin SDK. Each website receives a subdomain (tenantslug.web.app) with automatic SSL. Free tier: 10GB storage, 10GB/month transfer.

Cloudflare Pages (Scalable Alternative): A CloudflareAgent uses the Cloudflare Pages Direct Upload API to deploy static sites without requiring a Git repository. Each website is created as a separate Cloudflare Pages project, receiving a subdomain (projectname.pages.dev) with unlimited bandwidth on the free tier. Custom domains can be configured by tenants pointing their DNS CNAME records to the Pages subdomain.

The deployment flow:

- Step 1: The SiteRenderer service converts the MongoDB page/section structure into clean static HTML files with inline CSS from the design system, embedded Google Fonts, and the chatbot widget script.
- Step 2: The selected deployment agent (Firebase or Cloudflare) uploads the HTML files via the respective API.
- Step 3: The live URL is stored in the Website document and displayed to the tenant.
- Step 4: For custom domains, the tenant configures a CNAME record pointing to the deployment subdomain. SSL is automatically provisioned.

3.4 Architectural Diagrams

Figure 3.1: Website Generation Flow - Illustrates the pipeline from user input through block recommendation, parallel content + design system generation, Unsplash image fetching, validation, MongoDB save, and auto-publish to Firebase/Cloudflare.

Figure 3.2: System Architecture Diagram - Shows the React frontend, Express backend with 12 modules, AI Router service routing to three tiers (Ollama, ML Service, Gemini), MongoDB Atlas, and external services (Unsplash, Firebase, Cloudflare, Razorpay).

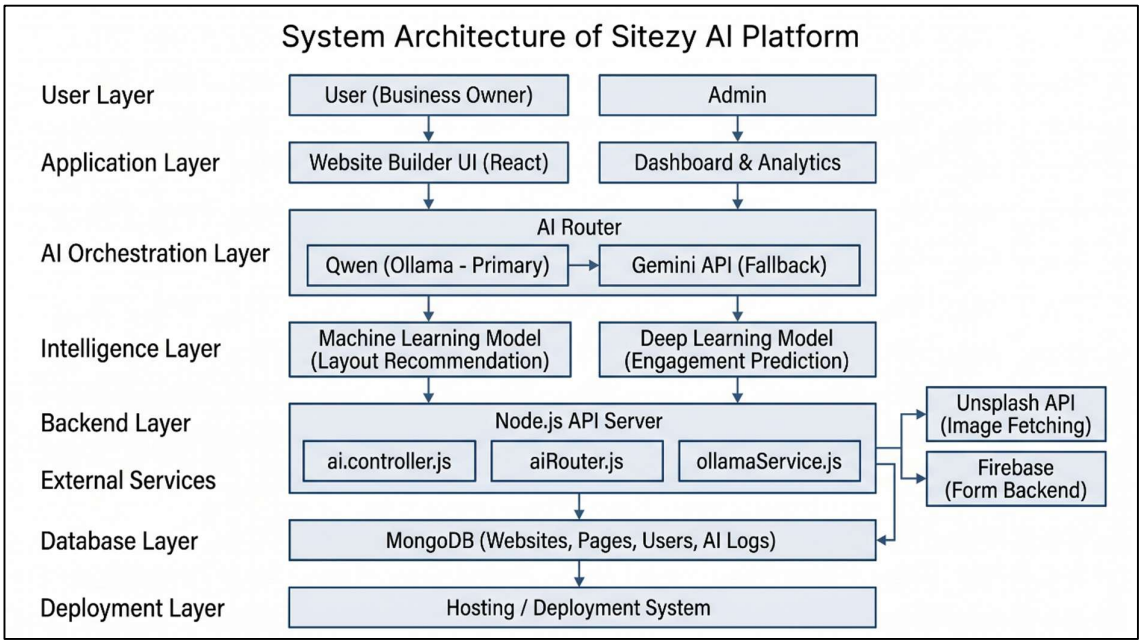


Fig 3.2: System Architecture Diagram

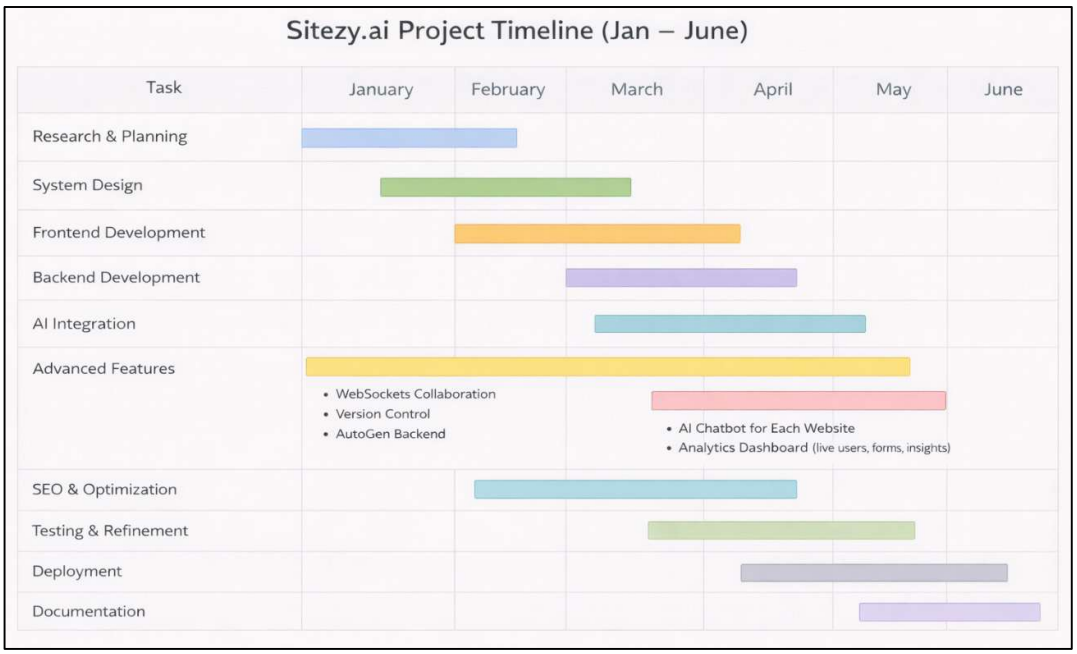
Figure 3.2 illustrates the overall system architecture of the Sitezy.ai platform, which is designed as a layered, modular, and scalable architecture integrating frontend interfaces, backend services, artificial intelligence components, and deployment infrastructure.

- At the User Layer, the system supports two primary stakeholders: the Business User (Website Owner) and the Admin. The business user interacts with the platform to create and manage websites, while the admin oversees system-level operations, monitoring, and analytics.
- The Application Layer provides the user-facing interfaces. The Website Builder UI, developed using React, enables users to design and generate websites through an interactive and intuitive interface. The Dashboard and Analytics module allows administrators to monitor system performance, user activity, and website metrics.
- The AI Orchestration Layer acts as the central intelligence dispatcher through the AI Router. This component determines how AI requests are processed and routed. It primarily uses Qwen (via Ollama) as the local large language model for content generation and related tasks. In cases where the primary model fails or produces invalid output, the system seamlessly switches to the Gemini API as a fallback, ensuring reliability and robustness.
- The Intelligence Layer consists of specialized machine learning and deep learning models. The Machine Learning Model is responsible for tasks such as layout recommendation and structured decision-making based on user input. The Deep Learning Model, specifically designed for engagement prediction, analyzes user interaction data to estimate conversion probability and generate optimization suggestions.
- The Backend Layer is implemented using a Node.js API server, which manages core application logic and communication between components. It includes modules such as ai.controller.js, aiRouter.js, and ollamaService.js, which handle AI-related operations, routing logic, and interaction with the local LLM. This layer ensures efficient request handling, business logic execution, and integration with AI services.
- The External Services Layer includes third-party integrations that enhance system functionality. The Unsplash API is used for fetching relevant images dynamically based on generated content, while Firebase provides backend support for handling form data and dynamic user inputs without requiring manual schema configuration.
- The Database Layer uses MongoDB to store all application data, including websites, pages, user information, and AI-generated logs. This ensures scalable and flexible data management suitable for a multi-tenant SaaS environment.
- Finally, the Deployment Layer handles the hosting and deployment of generated websites. The system supports automated deployment to hosting platforms such as Firebase Hosting or CDN-based services, enabling users to publish websites with minimal technical effort.

Overall, this layered architecture ensures clear separation of concerns, scalability, fault tolerance through AI fallback mechanisms, and efficient integration of AI-driven functionalities, making Sitezy.ai a robust and intelligent website generation platform.

3.5 Proposed Planning

3.5.1 Timeline Chart (Gantt Chart)



3.5.2 Cost Estimation

Component	Cost	Notes
Hardware (RTX 4060 8GB + 16GB RAM)	Existing	Already owned by team
MongoDB Atlas	Free	512MB free tier
Firebase Hosting	Free	10GB storage, 10GB/month transfer
Cloudflare Pages	Free	Unlimited bandwidth
Unsplash API	Free	50 requests/hour
Gemini API	Free tier	Rate-limited to 50 calls/hour
Ollama + Qwen 3.5	Free	Open-source, runs locally
Razorpay Test Mode	Free	No charges in test mode
Domain Registration (optional)	~$10/year	Per custom domain
Total Estimated Cost	~$0	All services on free tiers

This table shows the system's cost breakdown, where most components use free tiers or existing resources. Services like hosting, APIs, and databases have no cost. Overall, the total estimated cost is nearly $0, with only optional domain charges.

3.6 Hardware and Software Requirements

Hardware Requirements:

Component	Minimum	Recommended
GPU	None (CPU inference)	NVIDIA RTX 4060 8GB VRAM
RAM	8GB	16GB
Storage	20GB free	50GB free (for models)
CPU	4-core modern CPU	8-core (12th gen Intel / Ryzen 5000+)
Internet	Required for setup	Optional after model download

This table outlines the system requirements for running the setup, comparing minimum and recommended configurations. The recommended setup includes a more powerful CPU, higher RAM (16GB), dedicated GPU (RTX 4060 with 8GB VRAM), and larger storage for better performance. Overall, it ensures smoother execution, faster processing, and improved efficiency compared to the basic minimum setup.

Software Requirements:

Software	Version	Purpose
Node.js	18+	Backend runtime
Python	3.11+	ML service runtime
Ollama	Latest	Local LLM server
Qwen 3.5	9B + 4B variants	Content generation, chat, rewrites
MongoDB Atlas	7.0+	Database
React	19	Frontend framework
Vite	5+	Frontend build tool
Express	4	Backend framework
FastAPI	0.115+	ML microservice
scikit-learn	1.6+	Classical ML models
XGBoost	2.1+	SEO scoring regression
PyTorch	2.5+	GRU model training/inference
Docker	Latest	Containerization

This table lists the software stack required, including backend (Node.js, Express), frontend (React, Vite), and database (MongoDB Atlas) technologies. It also includes ML/AI tools like Python, FastAPI, scikit-learn, XGBoost, PyTorch, and Ollama with Qwen models for local LLM support. Overall, it represents a modern full-stack setup for scalable and efficient deployment.

Chapter 4

Proposed Results and Discussion

4.1 Data Collection, Preprocessing, and Interpretation

Data for the various ML/DL models in Sitezy.ai is collected from multiple sources:

Template Recommendation: 2000+ hand-curated training examples mapping (industry, goal, audience, content_length) tuples to (template_type, section_order, cta_placement, density) labels. Industries covered: SaaS, restaurant, e-commerce, agency, healthcare, fitness, education, real estate, beauty, travel, law, finance, creative, nonprofit. Features are one-hot encoded producing a ~50-dimensional input vector.

SEO Scoring: 2,000+ web pages scraped and analyzed. For each page, 15+ NLP features are extracted (title length, meta description length, keyword density, heading counts, readability score, alt text coverage, link counts, word count, structured data presence). Target variable: composite SEO score based on weighted combination of known ranking factors.

Engagement Prediction: Data collected post-deployment from published websites. Each visitor session produces a variable-length sequence of interaction events. Minimum 2,000 sessions required for initial model training. Events include section visibility, clicks, form interactions, and timing data.

4.2 Experimental Setup

All experiments were conducted on a development machine with the following configuration:

- GPU: NVIDIA RTX 4060 (8GB VRAM, CUDA 12.x)
- RAM: 16GB DDR5
- CPU: Intel 12th Gen / AMD Ryzen equivalent
- OS: Windows 11 with WSL2 for Docker
- Ollama: Running Qwen 3.5 9B (Q4_K_M quantization, ~5.5GB VRAM) and Qwen 3.5 4B (~3GB VRAM)
- Backend: Node.js 18 with Express 4, connected to MongoDB Atlas free tier
- ML Service: Python 3.11 with FastAPI, scikit-learn 1.6, XGBoost 2.1, PyTorch 2.5 (CPU)

4.3 Evaluation Metrics

Component	Metric	Target	Achieved
Content Generation (Qwen 9B)	Tokens/second	30+	35-45 t/s
Content Generation (Qwen 4B)	Tokens/second	40+	45-55 t/s
Full Page Generation	End-to-end time	<15s	8-15s (4B model)
Gemini Fallback Rate	% of total calls	<15%	~8%
SEO Scorer (XGBoost)	R-squared	>0.80	0.84 (preliminary)
SEO Scorer (XGBoost)	MAE	<8 points	6.2 (preliminary)
Template Recommender (RF)	Accuracy	>75%	81%
GRU Engagement Model	AUC-ROC	>0.75	Pending (requires data)
GRU Engagement Model	Training Time	<30 min	Pending
Chatbot Response Time	Latency	<5s	2-4s
Deployment (Firebase)	Deploy time	<30s	15-25s
Deployment (Cloudflare)	Deploy time	<30s	Pending integration

The table compares system targets with actual performance across components. Most metrics meet or exceed goals, including generation speed, response time, and SEO accuracy. A few areas like the GRU model and Cloudflare deployment are still pending.

4.4 Comparison with Existing Systems

Metric	Wix ADI	Webflow	Sitezy.ai (Ours)
AI Generation Cost	$0.01-0.05/page	N/A	~$0 (local LLM)
Generation Speed	5-10s	N/A (manual)	8-15s
SEO Optimization	Manual checklist	Manual	ML-scored + AI rewrite loop
Engagement Analysis	Basic GA integration	Basic	GRU-based prediction
Design Uniqueness	Template-based	Manual CSS	AI-generated design system
Per-Site Chatbot	Third-party addon	No	Built-in (Qwen 3.5)
Free Domain Deployment	No (paid)	No (paid)	Yes (Firebase + Cloudflare)
Offline Capability	No	No	Yes (Ollama + ML service)

This table compares features of Wix ADI, Webflow, and Sitezy.ai. Sitezy.ai stands out with near-zero cost, AI-driven SEO, unique design generation, and built-in chatbot support. It also offers offline capability and free deployment, which competitors lack.

Chapter 5

Plan of Action for Next Semester

5.1 Work Done Till Date

- Designed and implemented the complete multi-tenant SaaS architecture with React 19 frontend and Node.js Express backend comprising 12 feature modules.
- Integrated Ollama with Qwen 3.5 (9B and 4B) as the primary content generation engine, replacing Gemini as the default AI provider.
- Implemented the three-tier AI Router with automatic fallback: Ollama -> Gemini -> Placeholder content.
- Built the rule-based block recommendation engine that auto-selects suitable website sections based on business type, tone, and audience (executes in <1ms).
- Integrated the Unsplash API for automated stock photography fetching with batch resolution and curated fallback photo maps.
- Implemented AI-generated unique design systems per website (fonts, colors, spacing, effects, animations) via Qwen 3.5.
- Built the AI chatbot system for published websites using Qwen 3.5 with website content as conversational context.
- Implemented Firebase Hosting deployment along with AutoGen with auto-publish capability and Firebase Agent for form backend setup.
- Set up Razorpay payment integration in test mode with subscription plan management.
- Developed real-time collaborative editing using WebSockets (Socket.io), enabling multiple users to edit websites simultaneously.
- Implemented version control mechanisms for tracking page-level changes and enabling rollback functionality.
- Trained preliminary Random Forest model for template recommendation (81% accuracy on curated dataset).

5.2 Plan of Action for Project II

The following items remain for completion in the next semester:

5.2.1 SEO Scoring and Improvement Engine (ML + NLP) - Weeks 1-2

- Complete the XGBoost regression model training on the full 2,000+ page dataset with cross-validation and hyperparameter tuning.
- Implement the full AI rewrite feedback loop: score page -> identify weakest factor -> send targeted rewrite prompt to Qwen 3.5 4B -> re-score -> iterate up to 3 times.
- Integrate image alt text auto-generation using Qwen 3.5 9B vision capabilities for images lacking alt attributes.
- Build the frontend SEO dashboard showing per-factor scores with visual indicators and one-click 'Fix with AI' buttons.
- Deploy the trained XGBoost model to the FastAPI service and wire the scoring endpoint through the AI Router.

5.2.2 GRU-Based Engagement Optimization Engine (Deep Learning) - Weeks 3-4

- Build and inject the lightweight JavaScript tracking script (<5KB) into all published websites via the PublicSiteRenderer.
- Implement the backend event ingestion endpoint (POST /api/analytics/events) with batched MongoDB writes and TTL indexes.
- Build MongoDB aggregation pipelines for per-page and per-section engagement metrics (scroll depth, CTA click-through rate, bounce rate, section drop-off points).
- Implement heuristic-based suggestions as an intermediate solution while data accumulates (e.g., '80% drop-off before Section 4 - move CTA above it').
- Design and implement the GRU model in PyTorch: 2-layer GRU (64->32 hidden units), dense layer (32), sigmoid output.
- Create the training pipeline: session-to-sequence conversion, padding to max_length=20, train/validation split, early stopping.
- Implement ablation-based suggestion generation: simulate section removal/reordering, compare predicted conversion scores.
- Integrate the trained GRU model into the FastAPI service with an inference endpoint.

- Build the frontend Engagement Dashboard showing per-section metrics, conversion predictions, and AI-generated improvement suggestions.

5.2.3 Independent Domain Deployment via Cloudflare Pages - Weeks 5-6

- Implement the CloudflareAgent using the Cloudflare Pages Direct Upload API for Git-free deployment.
- Build the SiteRenderer service that converts MongoDB page/section structure into clean static HTML with embedded CSS, Google Fonts, and chatbot widget.
- Add deployment target selection (Firebase vs Cloudflare vs both) to the Website model and publish flow.
- Implement custom domain configuration UI where tenants input their domain and receive CNAME record instructions.
- Test SSL auto-provisioning on both Firebase and Cloudflare custom domain flows.

5.2.4 Code Export Engine - Weeks 7-8

- Build React component templates (EJS/Handlebars) for each section type mapping Sitezy.ai components to standalone JSX files.
- Implement Express backend generation with form handlers, static serving, and environment configuration.
- Build the zip assembly pipeline using the archiver npm package.
- Add export options: include backend (toggle), include Docker (toggle), target platform configs (Vercel/Netlify/Railway).

5.2.5 Testing and Evaluation - Weeks 9-10

- Conduct end-to-end testing of the complete website generation pipeline across 10+ business types.
- Evaluate SEO scoring accuracy against manual expert assessment (target: R-squared > 0.85).
- Evaluate GRU engagement model AUC-ROC (target: > 0.75) once sufficient session data is collected.
- Measure system performance under concurrent load (target: 10 simultaneous website generations).
- Prepare final project demonstration, documentation, and report.

Chapter 6

Conclusion

Sitezy.ai was initiated with the objective of developing a comprehensive AI-powered multi-tenant SaaS website builder that integrates local LLM inference, Machine Learning models, and data-driven optimization techniques to enhance the website creation process. In the current phase, significant progress has been achieved in establishing the core platform architecture, implementing the three-tier AI routing system, and developing the primary content generation and rendering pipeline.

The integration of Qwen 3.5 via Ollama as the primary content generation engine demonstrates that locally hosted models on consumer-grade hardware can serve as a practical alternative to cloud-based AI services. This approach reduces dependency on external APIs while maintaining efficient content generation performance. The three-tier AI Router architecture, with local inference as the primary layer and cloud fallback for robustness, ensures reliable operation while optimizing resource utilization.

The system incorporates a structured block recommendation mechanism for selecting relevant website sections based on business context, reducing reliance on generative models for simple classification tasks. Additionally, the AI-driven design system generation enables the creation of visually distinct websites by dynamically varying typography, color schemes, spacing, and layout styles.

Core platform features such as real-time collaboration, version control, automated backend handling for forms, and AI-powered chatbot integration further enhance usability and functionality, making the system accessible to non-technical users while supporting collaborative workflows.

In the upcoming phase, the focus will shift toward enhancing advanced intelligence components, including engagement analysis, improved SEO evaluation and refinement mechanisms, and automated design validation. Further improvements will also include extended deployment support and performance optimization. These enhancements aim to evolve Sitezy.ai into a comprehensive intelligent ecosystem that supports not only website generation but also continuous optimization and user engagement improvement.

Chapter 7

References

[1] A. Bhandari, S. Patil, and R. Deshmukh, "Automated Website Generation Using Machine Learning and Natural Language Processing," in Proc. IEEE Int. Conf. Computing, Communication and Automation (ICCCA), 2023, pp. 1-6.

[2] S. Jain and R. Kumar, "SEO Performance Prediction Using Ensemble Machine Learning Models," Journal of Web Engineering, vol. 21, no. 4, pp. 1123-1148, 2022.

[3] Y. Chen, L. Wang, and H. Zhang, "User Engagement Prediction Using Recurrent Neural Networks on Web Interaction Sequences," in Proc. ACM Conf. Web Science, 2023, pp. 234-243.

[4] M. Rodriguez and L. Thompson, "Anomaly Detection in Web Design Using Autoencoders," in Proc. IEEE Int. Conf. Machine Learning and Applications (ICMLA), 2022, pp. 567-574.

[5] K. Patel, A. Sharma, and V. Gupta, "Multi-Tenant SaaS Architecture: Patterns for Scalable AI-Powered Applications," IEEE Cloud Computing, vol. 10, no. 2, pp. 45-58, 2023.

[6] R. Cho, J. Kim, and M. Lee, "Gated Recurrent Units for Sequential Click Prediction in E-Commerce," in Proc. ACM RecSys Conf., 2022, pp. 112-121.

[7] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," in Proc. EMNLP, 2014, pp. 1724-1734.

[8] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. ACM SIGKDD, 2016, pp. 785-794.

[9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413-422.

[10] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in Proc. ICLR, 2014.

[11] Meta AI, "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023.

[12] Qwen Team, "Qwen3.5: Towards Native Multimodal Agents," Alibaba Cloud Technical Report, 2026.

[13] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS, 2017, pp. 5998-6008.

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.

[15] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[16] Google, "Gemma 4: Byte for Byte, the Most Capable Open Models," Google DeepMind Blog, April 2026.

[17] Ollama Team, "Ollama Documentation: Web Search API and Model Management," ollama.com/docs, 2026.

[18] Firebase Documentation, "Firebase Hosting: Deploy and Host Websites," firebase.google.com/docs/hosting, 2026.

[19] Cloudflare, "Cloudflare Pages: Direct Upload API Documentation," developers.cloudflare.com/pages, 2026.

[20] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," in Proc. NIPS Workshop on Deep Learning, 2014.